Laboratório Prolog · Aula 13

VITOR HUGO ASSIS DA SILVA · RA 241400232 · Turma B
Gerado em 20/10/2026, 20:21:03 · tempo registrado no cronômetro: 30:45

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

A consulta paradigma(L, funcional) responderá L = scheme, pois Scheme está cadastrado como uma linguagem funcional. A consulta paradigma(java, logico) responderá false, pois não existe na base um fato que relacione Java ao paradigma lógico.

Saída do ola.pl

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual é o paradigma de Prolog e usa a variável P para descobrir a resposta, que será P = logico. Já paradigma(prolog, logico) apenas verifica se o fato é verdadeiro, respondendo true ou false.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

A resposta false significa que o programa não conseguiu provar que Java é lógico. Pela hipótese do mundo fechado, o Prolog trata como falso tudo o que não está declarado ou não pode ser deduzido. Isso não prova uma verdade universal sobre Java, apenas representa o conhecimento disponível na base.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas: lucia, marta, tereza, joana, bia, ana e dora. A consulta (mae(M,F), mulher(F)) terá os pares (lucia,tereza), (marta,joana), (tereza,bia) e (joana,ana).

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
mae(joana,jake)
?- mae(jake,joana).
false.
?- pai(maria,jake).
false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
mae(joana,jake)
```

```
mae(joana,ana)
?- pai(A,jake).
pai(jorge,jake)
?- mulher(A).
mulher(lucia)
mulher(marta)
mulher(tereza)
mulher(joana)
mulher(bia)
mulher(ana)
mulher(dora)
--- 3. conjuncao
?- mae(joana,A),homem(A).
mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
mae(lucia,tereza),mulher(tereza)
mae(marta,joana),mulher(joana)
mae(tereza,bia),mulher(bia)
mae(joana,ana),mulher(ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta procura uma pessoa que seja mãe de uma filha do sexo feminino. Ela teve 4 respostas: lucia e tereza; marta e joana; tereza e bia; joana e ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

Isso acontece por causa da hipótese do mundo fechado: se algo não pode ser provado com os fatos e regras disponíveis, o Prolog considera falso. Isso pode ser problemático porque, em uma situação real, a ausência de informação não significa necessariamente que algo seja falso; a base pode estar incompleta.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

As respostas aparecem na ordem: lucia, marta, tereza, joana, bia, ana e dora. Essa ordem é determinada pela posição dos fatos no programa, pois o Prolog pesquisa de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

Previsão sobre tio e primo:
tio(X, jake) responde X = paulo. primo(leo, X) responde X = jake e depois X = ana.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falham f(X,X) = f(a,b), [A|B] = [] e 2 + 3 = 5. Somente f(X,X) = f(a,Y) unifica, ficando X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
avo(marta,jake)
avo(vern,jake)
?- avo(osvaldo,A).
avo(osvaldo,joana)
```

```
avo(osvaldo,paulo)
avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
irmao(jake,ana)
?- tio(A,jake).
tio(paulo,jake)
?- primo(leo,A).
primo(leo,jake)
primo(leo,ana)
--- ancestral
?- ancestral(osvaldo,A).
ancestral(osvaldo,vern)
ancestral(osvaldo,tereza)
ancestral(osvaldo,joana)
ancestral(osvaldo,paulo)
ancestral(osvaldo,jake)
ancestral(osvaldo,ana)
ancestral(osvaldo,leo)
ancestral(osvaldo,bia)
?- ancestral(A,jake).
ancestral(joana,jake)
ancestral(jorge,jake)
ancestral(lucia,jake)
ancestral(marta,jake)
ancestral(osvaldo,jake)
ancestral(vern,jake)
?- ancestral(jake,A).
false.

Resposta de unificacao.pl:

--- constantes e variaveis
a = a    unifica
a = b    NAO unifica
A = a    unifica   A = a
--- estruturas
f(A,b) = f(a,B)    unifica   A = a   B = b
f(A,A) = f(a,b)    NAO unifica
f(A,A) = f(a,B)    unifica   A = a   B = a
pai(A,jake) = pai(jorge,B)    unifica   A = jorge   B = jake
--- listas
[A|B] = [1,2,3]    unifica   A = 1   B = [2,3]
[A|B] = []    NAO unifica
[A,B|C] = [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3 = 5    NAO unifica
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

ancestral precisa de duas cláusulas para representar o caso direto e os casos com várias gerações. O caso base diz que X é ancestral de Y quando X é progenitor de Y. O caso recursivo diz que X é ancestral de Y quando X é progenitor de Z e Z é ancestral de Y.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, a consulta passa a considerar jake irmão dele mesmo. Assim, irmao(jake,X) responde jake e ana, enquanto antes respondia apenas ana. A condição é necessária para impedir que uma pessoa seja considerada

irmã de si própria.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X,X) = f(a,b), a primeira posição liga X a a, mas a segunda exigiria que o mesmo X fosse b, o que é impossível. Em f(X,X) = f(a,Y), X recebe a e a segunda posição liga Y ao mesmo valor. Portanto, X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Prolog, avo/2 descreve uma relação e os argumentos não possuem papéis fixos de entrada e saída. Por isso, a mesma regra pode descobrir avós ou netos. Em Java e Scheme, uma função normalmente recebe entradas em uma direção definida e devolve uma saída; seria necessário escrever outra função ou outro algoritmo para fazer a consulta inversa.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X,Y,[1,2,3]) terá 4 respostas:

X = [], Y = [1,2,3]

X = [1], Y = [2,3]

X = [1,2], Y = [3]

X = [1,2,3], Y = []

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
membro(b,[a,b,c])
?- membro(z,[a,b,c]).
false.
?- membro(A,[a,b,c]).
membro(a,[a,b,c])
membro(b,[a,b,c])
membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
tamanho([a,b,c],3)
?- tamanho([],A).
tamanho([],0)
--- concatena
?- concatena([1,2],[3,4],A).
concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
concatena([],[1,2,3],[1,2,3])
concatena([1],[2,3],[1,2,3])
concatena([1,2],[3],[1,2,3])
concatena([1,2,3],[],[1,2,3])
```

```
--- inverte
?- inverte([1,2,3],A).
inverte([1,2,3],[3,2,1])
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([],0). O passo recursivo calcula primeiro o tamanho da cauda e depois soma 1. Para [a,b,c], ficam pendentes três operações N is M + 1. Ao chegar à lista vazia, elas são resolvidas na volta, produzindo 1, 2 e finalmente 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Existem 4 respostas porque uma lista de três elementos pode ser dividida em duas listas em quatro posições: antes do primeiro elemento, depois do primeiro, depois do segundo e depois do terceiro. O anexar de Scheme normalmente apenas recebe duas listas e calcula a junção; ele não usa o resultado para descobrir todas as divisões possíveis.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Isso foi feito pela unificação combinada com recursão e backtracking. O Prolog primeiro tenta X como a cabeça da lista e, quando se pede outra resposta, volta e procura recursivamente na cauda.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 não calcula a soma: X fica ligado à estrutura 2+3. X is 2 + 3 calcula e produz X = 5. fatorial(N,120) gera erro de instanciação porque N está livre quando o programa tenta executar uma comparação ou operação aritmética.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
2+3=2+3
?- A is 2+3.
5 is 2+3
?- 5=:=2+3.
5=:=2+3
?- 2+3=5.
false.
?- A is 7 mod 3.
1 is 7 mod 3
?- A is 7//2.
3 is 7//2
?- A is 2^10.
1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
fatorial(5,120)
?- fatorial(0,A).
fatorial(0,1)
?- fib(10,A).
fib(10,55)
?- soma([1,2,3,4],A).
soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
```

```
ERRO: instantiation_error
?- A is B+1.
ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

O operador = apenas unifica termos, então X = 2+3 liga X à expressão sem calculá-la. O operador is calcula o lado direito, então X is 2+3 produz X = 5. Já =:= calcula os dois lados e compara os valores, portanto 5 =:= 2+3 é verdadeiro.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

fatorial(N,120) dá erro porque a regra tenta executar N > 0 e N1 is N-1 sem conhecer o valor de N. A aritmética exige operandos já definidos. concatena, por outro lado, é uma relação baseada em unificação e pode usar o terceiro argumento conhecido para descobrir o primeiro.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Os dois programas usam o mesmo padrão recursivo: caso base para zero e chamada com o número anterior. Em Haskell, fact é uma função que recebe um número e devolve um valor. Em Prolog, o resultado é colocado em um segundo argumento, mas o uso de is e das comparações faz com que essa implementação também funcione principalmente na direção em que N já é conhecido.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) dará false, enquanto solteiro1(X) encontrará osvaldo, paulo e jake. maximo2(5,3,M) terá duas respostas: M = 5 e M = 3. maximo3(5,3,3) dará true, embora matematicamente esteja errado.

Saída de controle.pl e mensagem de ancestral_ruim

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X,Y).
tenta ana e ana
tenta ana e bia
tenta ana e caio
resposta: mesmo_gosto(ana,caio)
tenta bia e ana
tenta bia e bia
tenta bia e caio
tenta caio e ana
resposta: mesmo_gosto(caio,ana)
tenta caio e bia
tenta caio e caio
--- B) negacao
?- solteiro1(A).
solteiro1(osvaldo)
solteiro1(paulo)
solteiro1(jake)
?- solteiro2(A).
false.
--- C) corte
?- maximo1(5,3,A).
maximo1(5,3,5)
?- maximo2(5,3,A).
maximo2(5,3,5)
```

```
maximo2(5,3,3)
?- maximo3(5,3,A).
maximo3(5,3,5)
?- maximo1(5,3,3).
false.
?- maximo3(5,3,3).
maximo3(5,3,3)

Resposta de laco_infinito.pl:

ancestral_bom(osvaldo, jake)?
sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores existem 24 colorações. Com 2 cores não existe nenhuma coloração, pois PR, SP e MS fazem fronteira entre si e formam um triângulo, exigindo três cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog escolhe os valores de X e Y na ordem dos fatos e testa se são diferentes e se gostam da mesma bebida. Em “ana e ana”, falha a meta X \== Y. Então ele volta ao último ponto de escolha, que é amigo(Y), e tenta o próximo valor para Y. As duas respostas encontradas são ana e caio, e caio e ana, pois ambos gostam de café.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em solteiro1, o Prolog primeiro escolhe um homem e só depois verifica se não consegue provar que ele é casado. Em solteiro2, a negação é executada com X ainda livre; como existe algum X casado, casado(X) tem sucesso e a negação falha antes de gerar os homens. Osvaldo aparece como solteiro porque não existe o fato casado(osvaldo). Ser pai de vern não permite concluir que ele é casado.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

A primeira cláusula de maximo3(5,3,3) falha na unificação da cabeça, pois o terceiro argumento deveria ser 5. Como o corte ainda não foi alcançado, o Prolog tenta a segunda cláusula, que aceita 3. Uma correção é fazer a unificação somente depois do corte:

maximo4(X,Y,M) :- X >= Y, !, M = X.
maximo4(_,Y,M) :- M = Y.

Assim, depois que X >= Y for confirmado, o corte impede que a segunda cláusula seja usada caso M = X falhe.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim chama a si mesmo antes de consumir qualquer relação de progenitor e repete a chamada indefinidamente, causando estouro da pilha. ancestral_bom procura primeiro um progenitor, avançando pela árvore familiar antes da chamada recursiva. Embora as regras tenham uma interpretação lógica semelhante, a busca em profundidade do Prolog é afetada pela ordem das metas.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

O programa encontrou 24 colorações com 3 cores, nenhuma com 2 cores e 1728 com 4 cores. Ele é declarativo porque o código descreve apenas as cores possíveis e a condição de que regiões vizinhas tenham cores diferentes. O mecanismo de backtracking do Prolog realiza a busca sem que seja necessário escrever os passos de um algoritmo de coloração.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma funcional, anexar recebe duas listas e devolve uma lista resultante. No paradigma lógico, concatena descreve uma relação entre três listas, podendo juntar listas ou usar o resultado conhecido para descobrir diferentes divisões.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Uma deficiência é a dependência da ordem das metas: ancestral_ruim entra em recursão infinita apenas porque a chamada recursiva aparece primeiro. Outra é a aritmética não ser totalmente relacional: fatorial(5,F) funciona, mas fatorial(N,120) gera erro porque N ainda não possui valor.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Eu escolheria Prolog para problemas baseados em relações, regras e busca, como sistemas especialistas, árvores genealógicas, consultas a bases de conhecimento, mapas e restrições. Não escolheria para aplicações com muito estado mutável, interfaces gráficas, processamento numérico intensivo ou controle detalhado da execução, situações em que Java é mais apropriado.